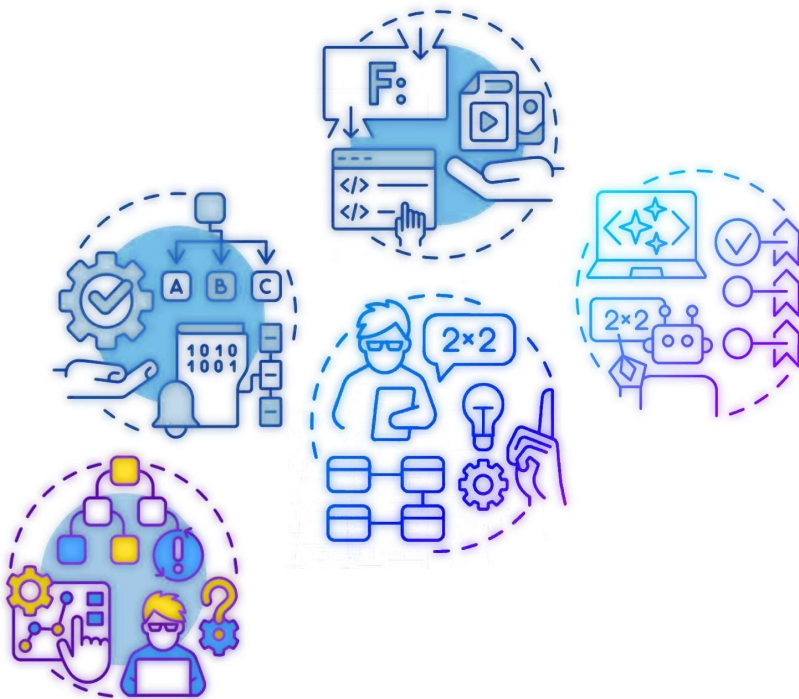




Paradigma Funcional

Esp. Ing. Viviana A. Santucci
AIA Federico Castoldi
Ing. V. Franco Matzkin
Ing. Cecilia Serafini

Tecnologías de la Programación

1 – Lambda-Cálculo

Es un sistema formal diseñado para investigar la definición de función, la noción de aplicación de funciones y la recursión. Es el más pequeño lenguaje universal de programación. Consiste de una regla de transformación simple (substitución de variables) y un esquema simple para definir funciones.

El cálculo lambda es universal porque cualquier función computable puede ser expresada y evaluada a través de él.

1 – Lambda-Cálculo

Por ejemplo, si una función f es definida por la ecuación $f(x)=t$,

donde t es algún término que contiene a x ,
entonces la aplicación $f(u)$ devuelve el valor $t[u/x]$,
donde $t[u/x]$ es el término que resulta de sustituir u en cada aparición
de x en t .

Por ejemplo, si $f(x)=x*x$, entonces $f(3)=3*3=9$.

1 – Lambda-Cálculo

- Lambda Expresiones

Church propuso una forma especial (más compacta) de escribir estas funciones. En vez de decir “la función f donde $f(x)=t$ ”, él simplemente escribió $\lambda x.t$.

Para el ejemplo anterior: $\lambda x.x*x$.

Un término de la forma $\lambda x.t$ se llama “**lambda expresión**”. La principal característica de lambda cálculo es su simplicidad ya que permite efectuar sólo dos operaciones: abstracción funcional y aplicación de función

1 – Lambda-Cálculo

Para la función:

$$f: A \rightarrow B$$

$$f(x) \ 2x + 1$$

Podemos escribirla como una expresión de la forma:

$$x . 2x+1$$

1 – Lambda-Cálculo

Para la función:

$$f(x, y) = (x + y) * 3$$

Se expresaría en Lambda-Calculo como:

$$x. y. * (+ x y) 3$$

Aplicar alguna de las funciones definidas sobre un argumento real (A); lo que es conocido también con el nombre de reducción, y que no es otra cosa que sustituir las ocurrencias del argumento formal (x), que aparezcan en el cuerpo (B) de la función, con el argumento real(A), es decir: (x.B) A

1 – Lambda-Cálculo

Ejemplo 3:

$$(x. (x+5)) 3$$

Lo que indica que en la expresión $x+5$, se debe sustituir el valor de x por 3.

Los dos mecanismos básicos presentados anteriormente se corresponden con los conceptos de abstracción funcional y aplicación de función; si le agregamos un conjunto de identificadores para representar variables se obtiene lo mínimo necesario para tener un lenguaje de programación funcional.

El cálculo Lambda tiene el mismo poder computacional que cualquier lenguaje imperativo tradicional.

2 – Evaluación Perezosa (Lazy)

Los lenguajes tradicionales, evalúan todos los argumentos de una función antes de conocer si éstos serán utilizados. Dicha técnica de evaluación se conoce como **evaluación ansiosa (eager evaluation)** porque evalúa todos los argumentos de una función antes de conocer si son necesarios.

Por otra parte, en ciertos lenguajes funcionales se utiliza **evaluación perezosa (lazy evaluation)** que consiste en no evaluar un argumento hasta que no se necesita.

Haskell, Miranda y Clean son perezosos, mientras que LISP, SML, Erlang y Scheme son estrictos.

2 – Evaluación Perezosa (Lazy)

Uno de los beneficios de la evaluación perezosa consiste en la posibilidad de manipular estructuras de datos 'infinitas'. Evidentemente, no es posible construir o almacenar un objeto infinito en su totalidad.

Sin embargo, gracias a la evaluación perezosa se puede construir objetos potencialmente infinitos pieza a pieza según las necesidades de evaluación.

3 – Racket - Datos atómicos

Los datos atómicos son el bloque de construcción básico de cualquier lenguaje de programación, y Racket no es una excepción.

Los datos atómicos se refieren a los tipos de datos elementales que generalmente se consideran entidades indivisibles; es decir, números como 123, cadenas de texto como “hola”, e identificadores como pi. Números y cadenas de texto.

Los números y las cadenas de texto se evalúan a sí mismos; si están vinculados, los identificadores se evalúan a su valor asociado.

3 – Racket - Datos atómicos

La evaluación de un identificador no vinculado resulta en un error. Para evitar que se evalúe un identificador no vinculado, entonces hay que prefijarlo con un apóstrofe.

```
> alpha
```

```
.. alpha: undefined;  
cannot reference an identifier before its definition
```

```
> 'alpha
```

```
'Alpha  
listas.
```

Se puede organizar datos atómicos juntos usando

4 – Racket - Sintaxis

Palabras clave, variables y los símbolos son llamados Identificadores. Los identificadores pueden estar formados por:

- [a-z]
- [A-Z]
- [0-9]
- ? ! . + - * / < = > : \$ % ° & _ ~ @
- Ejemplo: Hola, n, x, x3, ?\$&*!!!

4 – Racket - Sintaxis

- Todos los identificadores deben estar delimitados por:
 - Un espacio en blanco
 - Comillas dobles (“)
 - Paréntesis
 - carácter de comentario (;)
- No hay límite de longitud
- No es case-sensitive:
 - Abc, abc, aBc son todos el mismo identificador

4 – Racket - Sintaxis

- Las estructuras y las Listas se encierran entre paréntesis:
Ejemplo: (a b c) o (* (- x 2) y)
- () => Lista vacía
- Valores Booleanos:
#t → verdadero
#f → falso

4 – Racket - Sintaxis

- Vectores:
 - comienzan con #(
 - Finalizan con)
 - Ejemplo: #(esto es un vector de símbolos)
- String: encerrados en “”
- Caracteres: precedidos por #\
Ej: #\a

4 – Racket - Sintaxis

Números:

- Enteros: -123
- Racionales: $1/2$
- En punto flotante: 1.3
- Notación Científica: $1e12$
- Complejos en Notación Rectangular: $1.3-2.7i$
- Complejos en Notación Polar: $-1.2@73$

5 – Racket - Convención de Nombres

- Predicados finalizan en ?: retornan #t o #f
 - Ejemplo: eq?, zero?, string=?
- El nombre de la mayoría de los procedimientos de string, caracteres y vectores comienzan con el prefijo string-, char- y vector-
 - Ejemplo: string-append
- Conversión entre tipos de objetos se escriben como tipo1->tipo2.
 - Ej: vector->list

6 – Racket - Listas

Las listas son las principales estructuras de datos no atómicas (es decir, algo distinto a un número, cadena de texto, etc.). Racket se basa en gran medida en las listas porque es un descendiente de Lisp (abreviatura de LISt Processing). Aquí está cómo hacer una lista con algunos números:

```
>list( 1 2 3)
```

Observa la sintaxis. Las listas normalmente comienzan con un paréntesis abierto, (, seguido de una lista de elementos separados por espacios y terminan con un paréntesis cerrado,). El primer elemento de la lista es normalmente un identificador que indica cómo se debe evaluar la lista.

6 – Racket - Listas

Las listas pueden también contener otras listas:

```
>(list 1 (list "two" "three") 4 5)
```

Que se imprime como: `(1 ("two" "three") 4 5)`

el apóstrofe (o marca de tilde) al principio del último ejemplo es un alias para la palabra clave `'quote'`. Si quieres ingresar una lista literal (una lista que simplemente se acepta tal cual), puedes ingresarla entre comillas.

6 – Racket - Listas

```
> (quote (1 ("two" "three") 4 5))
```

Or

```
> '(1 ("two" "three") 4 5)
```

Cualquiera de los cuales se imprime como

```
'(1 ("two" "three") 4 5)
```

6 – Racket - Listas

Aunque 'list' y 'quote' parecen dos formas equivalentes de construir listas, hay una diferencia importante entre ellas. La siguiente secuencia ilustra la diferencia.

```
> (quote (3 1 4 pi))  
'(3 1 4 pi)
```

```
> (list 3 1 4 pi)  
'(3 1 4 3.141592653589793)
```

6 – Racket - Listas

‘quote’ devuelve la lista exactamente como se ingresó, pero cuando se usó ‘list’, el identificador pi fue evaluado y su valor fue sustituido en su lugar.

En general, en una lista no citada, todos los identificadores son evaluados y reemplazados por sus valores asociados.

La palabra clave ‘quote’ juega un papel importante en las macros y la evaluación de expresiones simbólicas.

6 – Racket - Listas

Quote ('') le dice a Racket que trate un identificador como símbolo y no como variable

- Los símbolos y variables en Racket son similares a los símbolos y variables en expresiones matemáticas y ecuaciones
- En expresiones Matemáticas:
 $1 - x \rightarrow$ pensamos en x como variable
- En expresiones Algebraicas:
 $X^2 - 2 \rightarrow$ pensamos en x como símbolo

6 – Racket - Listas

Operadores de Listas:

- `car`: retorna el primer elemento de la lista
 - `(car '(a b c)) ⇒ a`
- `cdr` (`could-er`): retorna el resto de la lista
 - `(cdr '(a b c)) ⇒ (b c)`
 - `(cdr '(a)) ⇒ ()`

6 – Racket - Listas

Operadores de Listas:

- `cons`: construye listas. Recibe dos argumentos. Usualmente el segundo es una lista y en ese caso retorna una lista.
 - $(\text{cons } 'a \ '()) \Rightarrow (a)$
 - $(\text{cons } 'a \ '(b \ c)) \Rightarrow (a \ b \ c)$
 - $(\text{cons } 'a \ (\text{cons } 'b \ (\text{cons } 'c \ '()))) \Rightarrow (a \ b \ c)$
 - $(\text{cons } '(a \ b) \ '(c \ d)) \Rightarrow ((a \ b) \ c \ d)$

6 – Racket - Listas

Cons: construye pares. Cuando el segundo parámetro es una lista devuelve una lista Propia.

- Lista Propia: Una lista vacía es una lista propia, y toda lista cuyo cdr sea una lista propia es una lista propia.
- Listas impropias: compuestas por pares donde se marca la separación de elementos por puntos.

7 – Racket - Expresiones y Notación prefija

Notación para la sintaxis de Racket.

Se utiliza la notación tipo BNF (Backus-Naur Form) que es una representación de gramáticas formales utilizada para describir la sintaxis de lenguajes de programación, protocolos de comunicación entre otros.

En Racket se puede utilizar la biblioteca **scribble/bnf** para representar gramáticas BNF.

7 – Racket - Expresiones y Notación prefija

- Todas las secuencias de caracteres delimitadas por `<` y `>` representan símbolos no terminales. Por ej. :
`<simbolo_no_terminal>`
- Todas las secuencias de caracteres no delimitadas, representan símbolos terminales. Por ej. `define`, `let`.
- El metaagrupamiento se hace con llaves `{` y `}`.
- El metasímbolo `+`, indica al menos una ocurrencia del símbolo precedente.
- El metasímbolo `*`, indica ninguna, una o varias ocurrencias del símbolo precedente.

7 – Racket - Expresiones y Notación prefija

Notación prefija.

Todas las expresiones tienen la forma ($\langle \text{operador} \rangle \langle \text{operando} \rangle^*$), es decir, que están siempre en notación prefija con pareamiento completo:

$(* 2 3)$ -> equivale a $(2 * 3)$

$(> 5 6)$ -> equivale a $(5 > 6)$

$(+ 2 3 10)$ -> equivale a $(2 + 3 + 10)$

$(+ 4 (* 3 2))$ -> equivale a $(4 + 3 * 2)$

Por ejemplo, la expresión infija $5a+2bc^2$ es: $(+ (* 5 a) (* 2 b c c))$.

8 – Racket - Evaluación de Expresiones

(procedimiento arg1 arg2 ...)

- Buscar el valor de procedimiento
- Buscar el valor de arg1
- Buscar el valor de arg2
-
- Aplicar el valor de procedimiento a los valores de arg1, arg2, ...

8 – Racket - Evaluación de Expresiones

Ejemplo:

(+ 3 4)

- el valor de + es el procedimiento adición
- el valor de 3 es el número 3
- el valor de 4 es el número 4
- aplicando el procedimiento adición a los números 3 y 4 devuelve 7